MODULE 2

PRACTICAL NO 1 : CLASSIFYING TRAFFIC SIGNS USING A STANDARD BENCHMARK IMAGE

```
import cv2
import numpy as np

def classify_traffic_signal(image_path):
    # Read the image using the path provided to the function
    img = cv2.imread(image_path)

    if img is None:
        return "Error: Image not found or cannot be read"

    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

    lower_red1 = np.array([0, 70, 50])
    upper_red1 = np.array([10, 255, 255])

    lower_red2 = np.array([170, 70, 50])
    upper_red2 = np.array([180, 255, 255])

    lower_yellow = np.array([15, 70, 50])
    upper_yellow = np.array([35, 255, 255])

    lower_green = np.array([40, 70, 50])
    upper_green = np.array([90, 255, 255])

    mask_red = cv2.inRange(hsv, lower_red1, upper_red1) + \
               cv2.inRange(hsv, lower_red2, upper_red2)

    mask_yellow = cv2.inRange(hsv, lower_yellow, upper_yellow)
```

```
    mask_green = cv2.inRange(hsv, lower_green, upper_green)
    red_score = cv2.countNonZero(mask_red)
    yellow_score = cv2.countNonZero(mask_yellow)
    green_score = cv2.countNonZero(mask_green)

    threshold = 50
    scores = {
        "RED": red_score,
        "YELLOW": yellow_score,
        "GREEN": green_score
    }
    dominant_color = max(scores, key=scores.get)

    if scores[dominant_color] > threshold:
        return f"Signal State: {dominant_color} (Score: {scores[dominant_color]})"
    else:
        return "Signal State: UNKNOWN/OFF"

# Give the full path of your image
print(classify_traffic_signal(r"D:/TYDS07/image1.PNG"))
```

Output :

